

Cascading Style Sheets (CSS)

HTML is just an skeletal layout of a website. We need CSS to design a website, add styles to it and make it look beautiful.

What is CSS

CSS stands for Cascading Style Sheet. CSS is optional but it converts an off looking HTML page into a beautiful and responsive website.

What is DOM?

DOM stands for document object model. When a page is loaded, the browser (i) creates a DOM of the page which is constructed as a tree of objects. (ii)

HTML id and class attribute

When an html element is given an id, it serves as a unique identifier for that element.

On the other hand, when an HTML element is given a class, it now belongs to that class. More than one elements

can belong to a single class but every element must have a unique id (if assigned).

$\swarrow$ unique id
`<div id = "first" class = "C1 C2 C3" >`
 $\swarrow$ multiple classes followed by spaces.

Three ways to add CSS to HTML

- (i) `<style>` tag $\rightarrow$ Adding `<style>...</style>`
- (ii) Inline CSS $\rightarrow$ Adding CSS using style attribute.
- (iii) External CSS $\rightarrow$ Adding a (.css) to HTML using `<link>` tag.

CSS Selector

A CSS selector is used to select an HTML elements for styling.

```
body {
```

```
    color : red ; → Declaration [property value]
```

```
    background : pink ;
```

```
}
```

Element Selector

It is used to select an element based off the tagname for example :

```
h2 {
```

```
    color : blue ;
```

```
}
```

Id Selector

It is used to select an element with a given id. For example :

Date: / /

first { → # is used to target an id.

```
color: white;
background: black;
}
```

Class Selection

It is used to select an element with a given class.

For Example :

```
.red {
```

```
background: red;
}
```

Important Notice

* We can group selectors like this :

```
h1, h2, h3, div {
```

```
color: red;
```

```
}
```


* We can use element selector like this:

p.red {

colour : red ;

}

* (*) can be used as a universal selector to select all the elements.

*

{

margin : 0 ;

padding : 0 ;

}

* An inline style will override external and internal styles.

The Color Property.

The CSS color property can be used to set the text color inside an element.

```

p {
  color : red;
}

```

Types of color values

(i) RGB → Specify color using red, green, blue values. eg. `rgb(200, 98, 70)`.

(ii) HEX Code → Specify color using HEX code eg. `#ff7f80`.

(iii) HSL → Specify the color using hsl values. eg. `hsl(8, 90%, 63%)`
 ↳ hue, saturation, lightness.

The background - colour property

The CSS color background property specifies the background color of the property.

Date : / /

```
.brown {  
    background-color : brown ;  
}
```

The background image property

Used to set an image as the background

```
body {  
    background-image : url ("123.jpg");  
}
```

The image is by default repeated in X and Y directions.

The background repeating property

Can be any of :

- * repeat - x → repeat in horizontal direction
- * repeat - y → repeat in vertical direction
- * no-repeat → image not repeat.

The background-size property

Can be following b-

cover → Fits and no empty space remains.

contain → Fits and image is fully visible.

auto → Display in original size.

{width} → Set width and height will be set automatically.

{{width}} {{height}} → Set width & height

The background-position property

Sets the starting position of an image.

.div {

background-position : left top ;

}

The background-attachment property

Defines a scrollable / non-scrollable character of a background image.


```
.div {
```

```
background-attachment : fixed ;
```

```
}
```

The background shorthand

A single property to set multiple background properties.

```
.div {
```

← colour

← image

← repeat

```
background : red url("23.jpg") no-repeat
```

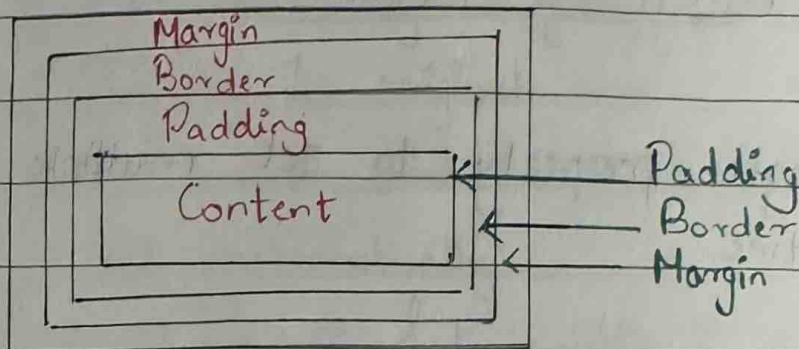
```
fixed right top ;
```

```
}
```

Date : / /

CSS Box Model

The CSS box model looks at all the HTML elements as box.

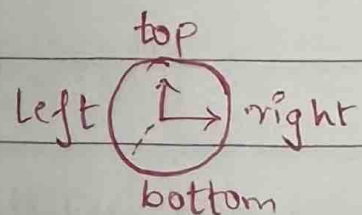


Setting width & height

We can set width & height in CSS as follows :-

```
#box {  
    height : 70 px  
    width : 70 px  
}
```

} Sets top, bottom, left and right values



Clockwise

Date: / /

• box Main {
top right bottom left
margin : 7px, 0px 2px 11px ;

• box Last {
top & bottom left and right
margin : 7px 3px ;

We can also set individual margins/paddings like this :

margin - top : 70px
margin - bottom : 3px
margin - left : 8px
margin - right : 9px
Same goes with padding.

Setting borders

We can set the borders as follows :-

• box {
border - width : 2px ;
border - style : solid ;
border - color : red ;
or just
border : 2px solid red ; [Shorthand]

Border Radius

We can set borders radius to create rounded borders

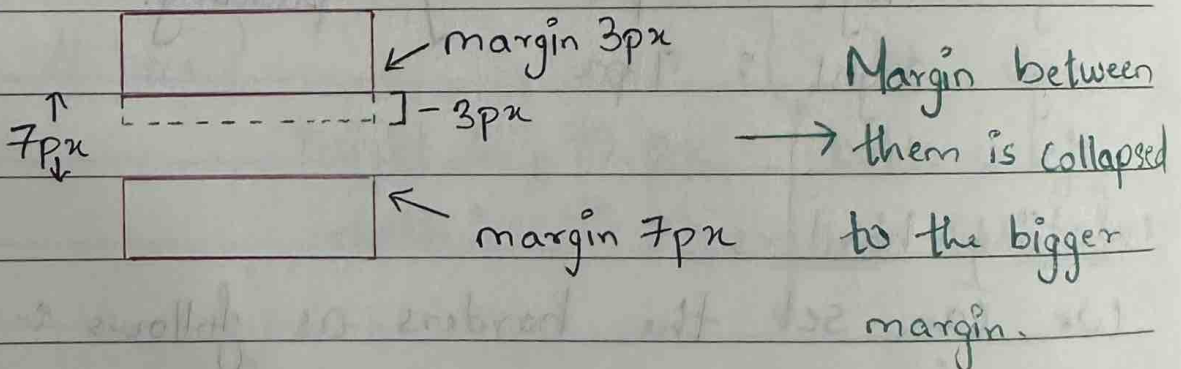
• div {

border-radius : 7px ;

}

Margin Collapse

When two margins from different elements overlap, the equivalent margin is the greater of two. This is called margin collapse.



Box Sizing

Determines what out of padding and border is included in elements width and height.

Can be content-box or border-box ←

↳ Include only
content in
width/height

• `div 1` {

`box-sizing : border-box ;`

}

↳ The content width
and height includes
Content + padding + border.

Fonts and Display

The display property

The CSS display property is used to determine whether an element is treated as a block/inline element and the layout used for its children.

flexbox / grid / etc

display : inline

Takes only the space required by the element. No linebreaks before and after. Setting width / height (or margin / padding) not allowed.

display : block

Takes full space available in width and leaves a newline before and after an element.

display : inline-block

Similar to inline but setting height, width, margin and padding is allowed. Elements can sit next to each other.

display : ~~none~~ ^{none} vs visibility : hidden

With display : none, the element is removed from the document flow. Its space is not blocked.

With visibility : hidden, the element is hidden but its space is removed.

Text-align property

Used to set the horizontal alignment of a text.

```
.div {  
    text-align : centre ;  
}
```

Text-decoration property

Used to decorate the text

Can be overline, line-through, underline, none

Text transform property

Used to specify uppercase and lowercase letters in a text.

p. uppercase {

text-transform : uppercase;

}

Line-height property

Used to specify the space between lines.

. small {

line-height : 0.7;

}

font

Font family specifies the plays a very important role in the look and feel of a website.

font-family

Font family specifies the font of a text.
Can hold multiple values as a "fallback" system.

P {

font-family: "Times New Roman", monospace;

, Always do this to ensure the correct font of your choice is rendered

Web Safe fonts

These fonts are universally installed across browsers.

Other font properties

Some of the other font properties are listed below :

font-size → Sets the size of the font

font-style → Sets the font style.

font-variant → Sets whether text is displayed in ^{Small} caps

font-weight → Sets the weight of the font.

Generic families

Broad class of similar fonts. eg serif, sans-serif.

Just like when we say fruit, it can be any fruit.

When we say serif, it can be any serif.

font-family → Specific

Generic family → Generic

Size, Position & Lists

There are more units for describing size other than 'px'. There are rem, em, vm, vh, percentage, etc.

What's wrong with pixels?

Pixels (Px) are relative to the viewing device for a device with size 1920×1080 , 1 px is 1 unit^{out} of $1080/1920$.

Relative lengths

These units are relative to the other length property. Following are some of the most commonly used relative lengths.

em → Unit relative to the parent font size.
↳ em means "my parent element's font size".

rem → Unit relative to root font size (<HTML> tag)

vm → Unit relative to 1% viewport width

vh → Unit relative to 100% viewport height

% → Unit relative to the parent element.

min/max - height/width property

CSS has a min-height, max-height, min-width, max-width property.

If the content is smaller than the minimum height, minimum height will be applied.

The position property

Used to manipulate the location of an element following are the possible values.

static → The default position. top/bottom/left/right / z-index

relative → The top/bottom/left/right / z-index will now work. Otherwise the element is in the flow of document like static

absolute → The element is removed from the flow and is relatively positioned to its first non-static ancestor.
top/bottom etc works.

fixed → Just like absolute except the element is positioned relative to the browser window.

Sticky → The element is positioned based on user's scroll position.

List-style property

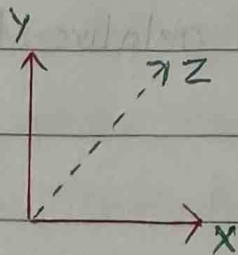
The list style property is a shorthand for type, position and image.

ul {
list style : square inside url('123.jpg')
}
list-style type list-style position list-style image.

Z-index property

The z-index property specifies the stack order of an element.

It defines which layer will be above which in case of overlapping ~~pages~~ elements.



→ Z is the third dimension

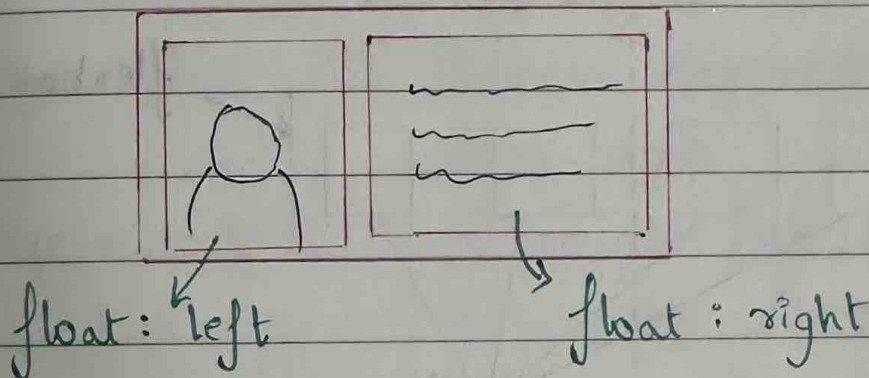
Date : / /

Flexbox

Before we look into the CSS flexbox, we will look into float and clear properties

The float property

Float property is simple. It just flows the element towards left/right



The clear property

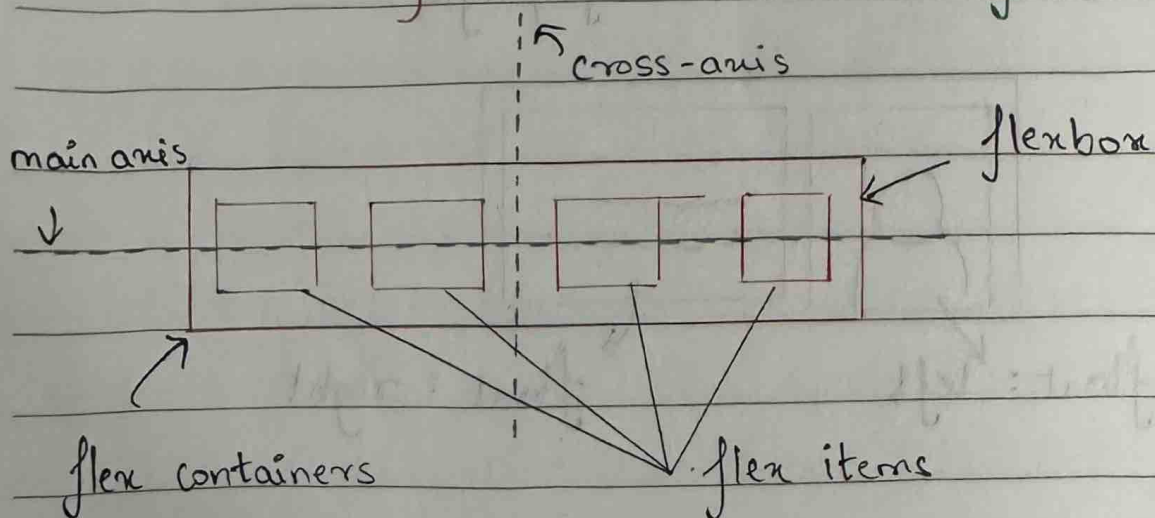
Used to clear the float. It specifies what elements can float beside a given element.

The CSS flexbox

Aims at providing a better way to layout, align, and distribute space among items in a container.

• Container {

display : flex ; $\Rightarrow$ Initialize a flexbox.



flex direction property

Defines the direction towards which items are laid. Can be row, row-reverse, column, column reverse.

Date : / /

flex properties for parent (flex container)

Following are the properties for the flex parent :

flex-wrap → Can be wrap, nowrap, wrap-reverse. Wrap items as needed with this properties

justify-content → Defines alignment along main axis.

align-items → Defines alignment along cross axis.

align-content → Aligns a flex container's lines when there is extra space in the cross axis.

flex properties for the children (flex items)

Following are the properties for the flex children :

Order → Controls the order in which the items appear in the flex container.

align-self → Allows default alignment to be overridden for the individual flex items.

flex-grow → Defines the ability for a flex item to grow.

flex-shrink → Specifies how much a flex item will shrink relative to the ~~rest~~ rest of the flex items.

CSS Grids and Media Queries

A CSS grid can be initialized using :

```
. Container {  
    display : grid ;  
}
```

All direct children automatically becomes grid items.

The grid-column-gap property

Used to adjust the space between the columns of a CSS grid.

The grid-row-gap property

Used to adjust the space between the rows of a CSS grid.

The grid-gap property

Shorthand property for grid-row-gap and grid-column-gap.

• Container {

display: grid; ✓^{row} ✓^{column}

grid-gap: 40px 100px;

}

Note :- For a single value of grid-gap, both row and column gaps can be set in one value.

Following are the properties for grid container:

1. The grid-template-columns property can be used to specify the width of columns.

• Container {

display: grid;

grid-template-columns: 8px 12px auto;

}

2. The grid-template-rows property can be used to specify the height of each row.

. container {

display : grid ;

grid-template-rows : 70px 150px ;
}

3. The justify-content property is used to align the whole grid inside the container.

4. The align-content property is used to vertically align the whole grid inside the container.

Following are the properties for grid item :

1. The grid-column property defines how many columns an item will span.

. grid-item {

grid-column : 1/5 ;
}

2. The grid-row property defines how many rows an item will span.

3. We can make an item to start on column 1 and span 3 columns like this :

```
.item {
  grid-column : 1 / span 3 ;
}
```

CSS Media Queries

Used to apply CSS only when a certain condition is true.

Syntax :-

```
@media only screen and (max-width : 800px)
```

```
{
  body {
    background : red ;
  }
}
```

Transforms, Transitions & Animations

Transforms are used to rotate, move, skew or scale elements. They are used to create a 3-d effect.

The transform property

Used to apply a 2D or 3D transformation to an element.

The transform-origin property

Allows to change the position of transformed elements.

2D transforms → Can change x and y axis

3D transforms → Can change z-axis as well

CSS 2D transform methods

You can use the following 2-D transforms in CSS:

1. `translate()`

2. `rotate()`

3. `scaleX()`

4. `scaleY()`

5. `skew()`

6. `Matrix()`

7. `scale()`

CSS 3D transform methods

1. `rotateX()`

2. `rotateY()`

3. `rotateZ()`

CSS Transitions

Used to change property values smoothly, over a given duration.

The transition property.

The transition property is used to add transition in CSS.

Following are the properties used for CSS transition.

1. transition-property → The property you want to transition.

2. transition-duration → Time for which you want transition to apply.

3. transition-timing-function → How you want the property to transition.

4. transition-delay → Specifies the delay for the transition.

All these properties can be set using a single shorthand property.

transition : width 3s ease-in 2s ;

① property ② duration ③ timing-function ④ delay

Transitioning multiple properties

We can transition multiple properties as follows:

transition : opacity 1s ease-out 1s, transform 2s ease-in ;

→ Yes you can skip transition delay here!

CSS Animations

Used to animate CSS properties with more control. We can use @Keyframe rules to change the animation from a given style to a new style.

@Keyframes herry {
from { width : 20px ; } → Can change
to { width : 31px ; } multiple properties
}

Properties to add Animations

Following are the properties used to set animations in CSS.

1. animation-name → name of the animation.
2. animation-duration → How long does the animation run?
3. animation-timing-function → Determines speed curve of the animation

4. animation-delay → Delay for the start of an animation.

5. animation-iteration-count → Number of times an animation should run.

6. animation-direction → Specifies the direction of the animation.

The animation shorthand

All the animations properties from 1-6 can be applied like this :

animation 6 herry 6s linear 1s infinite reverse

Using percentage value states with animation.

We can use % values to indicate what should happen when a certain percent of animation is completed.

@ Keywords harry {

0% {

width : 20px ;

50% {

width : 80px ;

100% {

width : 200px ;